

---

## ◆ 1. Pipelines

---

### Overview:

A **pipeline** automates build, test, and deploy steps — usually triggered on events like push, merge, or tag creation.

---

### Interview Q&A:

**Q1: What is a pipeline in Bitbucket?** A1: It's a **YAML-defined CI/CD process** triggered by repository actions (e.g., push to `main` ).

---

**Q2: What are the common pipeline stages?** A2:

- `build`
  - `test`
  - `deploy` Each has **custom scripts, caches, environment variables**, and triggers.
- 

**Q3: When would you use multiple pipelines?** A3:

- Per branch (e.g., `main` , `develop` )
  - Per tag (for releases)
  - For specific directories or pull requests
- 

## ◆ 2. Bitbucket Pipelines

---

### Overview:

**Bitbucket Pipelines** is Atlassian's native CI/CD engine. It uses a `.bitbucket-pipelines.yml` file in the root of your repo.

---

## ✓ Example:

```
pipelines:
  default:
    - step:
        name: Build & Test
        image: node:18
        script:
          - npm install
          - npm test
```

---

## 💬 Interview Q&A:

Q1: How do you enable pipelines in Bitbucket? A1:

- Go to repo → **Pipelines tab** → Enable
- Commit `.bitbucket-pipelines.yml` at repo root

---

Q2: Can Bitbucket Pipelines deploy to AWS or DockerHub? A2: ✓ Yes — with integrations or CLI tools (e.g., `aws-cli`, `docker` ).

---

Q3: How do you pass secrets to pipelines? A3:

- Use **repository variables** or **secured project variables** under settings
- Access them via `$MY_SECRET` in YAML

---

## ◆ 3. Version Control

---

### 🔍 Overview:

Bitbucket is a **Git-based version control system**. It supports **branching strategies**, **commit history**, **PRs**, and **tagging**.

---

## 💬 Interview Q&A:

**Q1: How does Bitbucket differ from GitHub/GitLab?** A1: Bitbucket is deeply integrated with **Atlassian tools** (Jira, Trello, Confluence), and supports **Mercurial (legacy)**.

---

**Q2: What is the default branching model in Bitbucket?** A2:

- `main/master` : production
  - `develop` : integration
  - `feature/*` , `bugfix/*` , `release/*` , `hotfix/*`
- 

**Q3: Can you revert a merge in Bitbucket UI?** A3:  Yes — under **Commit history**, Bitbucket offers a "Revert" button if fast-forward merges are enabled.

---

## ◆ 4. Self-Hosted Runners

---

### Overview:

Instead of Bitbucket's cloud-hosted pipelines, you can run jobs on **your own infrastructure** (VM, EC2, Kubernetes, etc.).

---

### How It Works:

- Install Bitbucket runner agent
- Register it with your repo/project
- Use in `.bitbucket-pipelines.yml` via:

```
runs-on:  
  - self.hosted
```

---

### Interview Q&A:

**Q1: When would you use self-hosted runners?** A1:

- Custom dependencies (e.g., GPU, specific OS)

- Sensitive environments (VPN, on-prem servers)
  - Performance/cost optimization for long jobs
- 

**Q2: Can self-hosted runners be autoscaled? A2:**  With Kubernetes or CI agents that support auto-scaling based on queue length or resource usage.

---

**Q3: How are runners authenticated? A3:** Each runner uses a **UUID token** and **secret key** tied to the repo/project.

---

---

## ◆ 5. Code Review

---

### Overview:

Bitbucket supports structured code reviews via **Pull Requests (PRs)** where reviewers can leave inline comments, suggestions, and approve/decline changes.

---

### Interview Q&A:

**Q1: What is a code review in Bitbucket? A1:** A **formal process** of reviewing code in a pull request before merging it into a main branch.

---

**Q2: What features support effective code reviews? A2:**

- Inline comments
  - Required approvers
  - Change request mechanism
  - Diff views (side-by-side or unified)
  - Task list comments
- 

**Q3: How can you enforce quality via reviews? A3:**

- Enforce **minimum number of approvals**
  - Restrict merging without passing pipeline
  - Block merges if unresolved tasks exist
- 

## ◆ 6. Merge Requests

---

### Overview:

Bitbucket's **Pull Requests (PRs)** are the equivalent of **Merge Requests** in GitLab. They trigger the **merge process** and act as the foundation of collaborative development.

---

### Interview Q&A:

Q1: What happens when you create a PR in Bitbucket? A1:

- Compares source and target branches
  - Allows reviewer assignment
  - Triggers CI/CD pipelines (optional)
- 

Q2: Can you enforce merge rules? A2:  Yes — enforce **branch permissions**, **build status checks**, and **code approvals** before merge.

---

Q3: What are common merge strategies in Bitbucket? A3:

- Merge commit
  - Squash merge
  - Rebase and merge (configurable)
- 

## ◆ 7. Pull Strategies

---

### Overview:

A **pull strategy** defines how code from feature branches is integrated into main branches — Bitbucket supports Git's full capabilities here.

---

## Interview Q&A:

Q1: What's the difference between merge vs. rebase? A1:

- **Merge:** preserves full history, creates a merge commit
  - **Rebase:** rewrites history for linear commits
- 

Q2: What is **squash merging**? A2: Combines all commits in a PR into **one clean commit** on merge — useful for cleaner commit logs.

---

Q3: When should you avoid rebasing? A3: When working with shared branches — rebasing rewrites history and can cause conflicts for other collaborators.

---

## 8. Secrets Management

---

### Overview:

Bitbucket provides **secured environment variables** for **pipelines and runners**, without exposing them in logs or code.

---

## Interview Q&A:

Q1: Where can you define secrets in Bitbucket? A1:

- At **repository level** under **Repository Settings** → **Pipelines** → **Repository Variables**
  - Or at **workspace/project level**
- 

Q2: Are secrets encrypted at rest? A2:  Yes — Bitbucket encrypts all secrets and ensures they are masked in logs.

---

Q3: Can secrets be used in custom scripts? A3:  Yes — injected into the shell at runtime using `$MY_SECRET`, just like normal environment variables.

---

## ◆ 9. Issues Board

---

### Overview:

Bitbucket's Issues Board allows teams to **track tasks, bugs, features**, and link them to commits or pull requests — similar to Jira (but simpler).

---

### Interview Q&A:

Q1: What's the use of the Bitbucket Issues feature? A1: It provides a lightweight **issue tracker** where devs can log bugs, assign issues, add priorities, and close via commit messages.

---

Q2: How can you link issues to commits? A2: Include the issue ID in commit messages like:

```
git commit -m "Fixes #123: Handle login error"
```

---

Q3: Does Bitbucket support epics or sprints like Jira? A3:  No — but it can integrate with Jira for full agile workflow management.

---

## ◆ 10. Tagging

---

### Overview:

Git tags in Bitbucket mark **release points** (e.g., v1.0) and can be used to **trigger deployments**, rollback, or lock artifacts.

---

## ✓ Example:

```
git tag v1.0
git push origin v1.0
```

---

## 💬 Interview Q&A:

Q1: Difference between lightweight and annotated tags? A1:

- **Lightweight:** Just a name → pointer to a commit
  - **Annotated:** Includes metadata → author, message, date
- 

Q2: Can tags trigger pipelines in Bitbucket? A2: ✓ Yes — use:

```
pipelines:
  tags:
    "v*":
      - step: ...
```

---

Q3: Can you delete a tag from Bitbucket? A3: Yes — use:

```
git push origin :refs/tags/v1.0
```

---

## ◆ 11. CI Status Badges

---

### 🔍 Overview:

CI status badges show whether the **latest build passed or failed**, and can be embedded in README files or dashboards.

---

## ✓ Example (Markdown):



! **[Build Status]** (<https://bitbucket.org/<workspace>/<repo>/addon/pipelines/home#!/results>)

---

## Interview Q&A:

**Q1: Why are CI badges useful? A1:** They give **at-a-glance visibility** into pipeline status on README or documentation pages.

---

**Q2: Can Bitbucket generate badges automatically? A2:**  Yes — available under **Pipelines** → **Settings** → **Status Badge**.

---

**Q3: Do badges show specific branches? A3:** Yes — you can configure badges to show build status for `main`, `dev`, etc.

---

## 12. Fork Handling

---

### Overview:

Forking in Bitbucket creates a **copy of the repository** — commonly used in open-source or isolated feature development.

---

## Interview Q&A:

**Q1: When should you fork instead of branch? A1:**

- You don't have write access
  - You want to experiment separately
  - You're working in open-source
- 

**Q2: How do you sync your fork with upstream? A2:**

```
git remote add upstream https://bitbucket.org/upstream/repo.git
git fetch upstream
git merge upstream/main
```

---

Q3: Can you open PRs from a fork to the base repo in Bitbucket? A3:  Yes — just like GitHub, Bitbucket allows cross-repo PRs.

---

---

## ◆ 13. Projects Board

---

### Overview:

The **Projects Board** in Bitbucket allows teams to **group repositories**, apply **shared permissions**, and manage them as a unit — especially useful for monorepos or microservices.

---

### Interview Q&A:

Q1: What is a Bitbucket Project? A1: A **logical grouping** of repositories with common **access control** and **visibility settings**.

---

Q2: Can you apply CI settings project-wide? A2: Yes — things like **deployment variables**, **runners**, and **build configurations** can be inherited or replicated across repos.

---

Q3: Can you archive or delete a Bitbucket project? A3:  Yes — archived projects prevent edits but preserve code/history.

---

---

## ◆ 14. Git Tags

---

### Overview:

Tags in Git are used to **mark fixed points in history** like releases, versions, or snapshots.

---

### ✓ Example:

```
git tag -a v1.0.0 -m "First release"  
git push origin v1.0.0
```

---

### 💬 Interview Q&A:

Q1: What's the difference between branches and tags? A1:

- **Branches** move forward with commits
  - **Tags** are static references to specific commits
- 

Q2: How are tags used in CI/CD? A2:

- Trigger production builds
  - Deploy exact versions
  - Rollback via known commit state
- 

Q3: Can tags be edited or deleted? A3: ✓ Yes — but requires force push and caution:

```
git tag -d v1.0  
git push origin :refs/tags/v1.0
```

---

## ◆ 15. Code Reviews (Culture & Practice)

---

### 🔍 Overview:

This topic focuses on the **collaborative and quality assurance** aspect of reviews — beyond tooling (already covered).

---

## Interview Q&A:

Q1: What's a healthy code review process? A1:

- Clear ownership of components
  - Small PR sizes
  - Constructive, respectful feedback
  - Review time SLAs (e.g., respond within 24h)
- 

Q2: What should a reviewer look for? A2:

- Correctness
  - Security holes
  - Code style
  - Edge case handling
  - Test coverage
- 

Q3: What metrics can help improve code reviews? A3:

- Time to merge
  - Comments per PR
  - Review coverage per author
  - Review-to-release delay
- 

## 16. Wiki Management

---

### Overview:

Bitbucket offers a built-in **wiki per repository** (in Markdown), perfect for storing **README-style docs**, API usage, onboarding guides, etc.

---

## Interview Q&A:

Q1: How do you edit a Bitbucket Wiki? A1:

- Web editor via the **Wiki tab**
- Or clone the wiki repo:

```
git clone https://bitbucket.org/workspace/repo/wiki.git
```

---

**Q2: Can you version control the wiki? A2:**  Yes — it's a Git repo with its own commit history.

---

**Q3: How is a wiki different from a README? A3:**

- README = repo landing page
  - Wiki = multipage, navigable documentation system
- 
- 

## ◆ 17. Starred Projects

---

### Overview:

Starring repositories or projects lets you **bookmark them for quick access**, especially useful in large organizations or multi-project workspaces.

---

### Interview Q&A:

**Q1: What happens when you star a project/repo in Bitbucket? A1:** It gets added to your **personal quick-access list** on the Bitbucket dashboard. It doesn't affect team visibility or notifications.

---

**Q2: Can multiple users star the same project? A2:**  Yes — starring is user-specific and doesn't impact others.

---

**Q3: Does starring affect project ranking or sorting? A3:** Yes — starred items are often shown at the top of dashboards or lists.

---

## ◆ 18. Discussions

---

### Overview:

Bitbucket Discussions let team members comment on commits, lines of code, or general issues — similar to GitHub's PR conversations.

---

### Interview Q&A:

Q1: What types of discussions does Bitbucket support? A1:

- PR comments
  - Inline diff discussions
  - Task list conversations
  - Commit-level comments
- 

Q2: Can you resolve or delete discussion threads? A2:  Yes — threads can be resolved, marked complete, or deleted if unnecessary.

---

Q3: Are comments stored in Git history? A3:  No — they are stored on the Bitbucket platform, not in Git commits.

---

## ◆ 19. Actions Panel

---

### Overview:

The **Actions Panel** in Bitbucket shows quick options like **clone**, **create pull request**, **branch from commit**, etc., depending on context.

---

### Interview Q&A:

Q1: What can you do from the Actions Panel? A1:

- Clone repo
  - View file history
  - Create a new branch
  - Open a PR
  - Delete/move files (if permissions allow)
- 

Q2: Can you customize the actions shown? A2: ❌ Not directly — but they adapt to context (commit view vs file view).

---

Q3: Is the Actions Panel user-specific? A3: Partially — it responds to **user permissions** and current view but the layout is the same for all users.

---

## ◆ 20. Commit Graph

---

### 🔍 Overview:

Bitbucket shows a **visual commit graph** — a tree-style timeline of branches, merges, and commits.

---

### 💬 Interview Q&A:

Q1: What is the purpose of the commit graph? A1: It helps visualize **branch divergence**, **merge flow**, and **commit ancestry**.

---

Q2: Can the commit graph help spot merge conflicts early? A2: ✅ Yes — overlapping parallel branches or rebase gaps often hint at future conflicts.

---

Q3: Does Bitbucket support interactive rebase from the graph UI? A3: ❌ No — rebasing must be done locally using Git CLI.

---

---

## ✔ Bitbucket Combined Interview Questions & Answers

---

### ◆ 1. How do Bitbucket Pipelines differ from Jenkins or GitHub Actions?

**Answer:** Bitbucket Pipelines is a **built-in CI/CD** tool specific to Bitbucket, requiring no external setup. Unlike Jenkins (self-hosted, plugin-based) or GitHub Actions (workflow-based), Bitbucket Pipelines uses a `.bitbucket-pipelines.yml` file and integrates tightly with the repository UI, variable management, and deployment environments.

---

### ◆ 2. How can you enforce code quality before merging to `main` ?

**Answer:**

- Require **at least 1 approval** in Pull Request settings
  - Enable **required build status** (pipeline must pass)
  - Block merges with **unresolved PR tasks**
  - Use **branch restrictions** so only PRs can update `main`
- 

### ◆ 3. Describe a secure way to deploy secrets in a pipeline.

**Answer:**

- Use **repository or project-level variables**
  - Mark them as **secured** (masked in logs)
  - Access them in YAML like `script: - echo $API_KEY`
  - Never hardcode or echo directly in logs
- 

### ◆ 4. When would you choose to use a self-hosted runner instead of Bitbucket's default?

**Answer:**

- If builds need **custom OS, CPU/GPU, or private VPN access**
  - To avoid build limits and get **faster queues**
  - For **data privacy** or running on **air-gapped** networks
-



## ◆ 5. How does tagging help with deployments and rollback in Bitbucket?

**Answer:** Git tags (e.g., `v1.0.0` ) mark a commit for release. Pipelines can be triggered on tag push:

```
pipelines:  
  tags:  
    "v*": ...
```

Rollback is possible by redeploying a previous tag.

---

## ◆ 6. What's the difference between squash merge and rebase merge in PRs?

**Answer:**

- **Squash:** Combines all commits into one, cleans history
  - **Rebase:** Rewrites commit base to make history linear
  - Both are used for tidier commit logs, but **rebase rewrites history** and should be avoided on shared branches.
- 

## ◆ 7. How do you use Discussions in Bitbucket to enforce quality collaboration?

**Answer:** Use **inline comments** on PR diffs to raise issues. Turn comments into **tasks**, which can block merging until resolved. Discussions also document design decisions asynchronously.

---

## ◆ 8. How can you identify problematic branches or potential conflicts early?

**Answer:** Use the **commit graph** to visualize branch divergence. Parallel branches with overlapping commits or unmerged histories can hint at rebase/merge conflicts.

---

## ◆ 9. How do forks work in Bitbucket and when are they useful?

**Answer:** Forks are **isolated copies** of repositories. Use them when:

- Contributing to external/public code
  - You don't have write access
  - You want to isolate feature branches entirely You can open PRs from forks back into the main repo.
-

## ◆ 10. What does the Actions Panel allow you to do in Bitbucket?

Answer: It provides **contextual actions** like:

- Clone repo
  - Create PR from a branch
  - View file history
  - Start a new branch from a commit Useful for quick navigation without terminal use.
- 

## ◆ 11. How does Bitbucket's Projects Board help in large-scale DevOps?

Answer: It groups multiple repos under a **common namespace** for:

- Shared permissions
  - Centralized pipelines
  - Easier cross-repo navigation
  - Logical segmentation (e.g., frontend, backend, data)
- 

## ◆ 12. What best practices do you follow in Bitbucket code review process?

Answer:

- Keep PRs small and focused
  - Use **task lists** and resolve all discussions
  - Ensure **build passes before reviewing**
  - Apply consistent review templates or checklists
  - Approve only if tested locally or CI has passed
-